CS 170 Programming Quiz 8

For **PQ08 - Processing Data with Arrays**, you'll write two methods that use arrays. This exam covers the first part of Chapter 7 in your textbook. For this exam you'll need to know how to:

- correctly define a method that takes and returns arrays
- create initialized and sized (instantiated) arrays and array variables
- write methods that take and return arrays
- write procedures that modify array parameters
- write methods that process arrays using selection statements and/or the logical operators.

You will not need to process arrays with loops in this exam. As on the previous exam, each of the two methods will be tested for syntax correctness (name, public, return type and parameters) and for correctness. Work step-by-step so that you get partial credit. You can find more problems to practice with on Codingbat and on Practice It. You'll find the links on Canvas.

Note: please do not just memorize the answers to these problems. The actual problems on the exams are always variations of the problems from the problem set.

1 The Make Two Problem

Given 2 int arrays, a and b, return a new array length 2 containing, as much as will fit, the elements from a followed by the elements from b. The arrays may be any length, including 0, but there will be 2 or more elements available between the 2 arrays.

- `make2([4, 5], [1, 2, 3]) → [4, 5]`
- `make2([4], [1, 2, 3]) → [4, 1]`
- `make2([], [1, 2]) → [1, 2]`

2 THE FRONT PIECE PROBLEM

Given an int array of any length, return a new array of its first 2 elements. If the array is smaller than length 2, use whatever elements are present.

- `frontPiece([1, 2, 3]) → [1, 2]`
- `frontPiece([1, 2]) → [1, 2]`
- `frontPiece([1]) → [1]`

3 THE TRIPLE CROWN PROBLEM

Given an array of ints of odd length, look at the first, last, and middle values in the array and return the largest. The array length will be a least 1.

- `tripleCrown([1, 2, 3]) → 3`
- `tripleCrown([1, 5, 3]) → 5`
- `tripleCrown([5, 2, 3]) → 5`

4 THE MID THREE PROBLEM

Given an array of ints of odd length, return a new array length 3 containing the elements from the middle of the array. The array length will be at least 3.

- `midThree([1, 2, 3, 4, 5]) → [2, 3, 4]`
- `midThree([8, 6, 7, 5, 3, 0, 9]) → [7, 5, 3]`
- `midThree([1, 2, 3]) → [1, 2, 3]`

5 THE SWAP ENDS PROBLEM

Given an array of ints, swap the first and last elements in the array. Return the modified array. The array length will be at least 1.

- `swapEnds([1, 2, 3, 4]) → [4, 2, 3, 1]`
- `swapEnds([1, 2, 3]) → [3, 2, 1]`
- `swapEnds([8, 6, 7, 9, 5]) → [5, 6, 7, 9, 8]`

6 THE PLUS TWO PROBLEM

Given 2 int arrays, each length 2, return a new array length 4 containing all their elements.

- `plusTwo([1, 2], [3, 4]) → [1, 2, 3, 4]`
- `plusTwo([4, 4], [2, 2]) → [4, 4, 2, 2]`
- `plusTwo([9, 2], [3, 4]) → [9, 2, 3, 4]`

7 THE PLUS TWO PROBLEM

Given 2 int arrays, each length 2 or greater, return a new array length 4 containing the first two elements from a followed by the last two elements from b.

- `plusTwo([1, 2, 3, 4], [5, 3, 4]) → [1, 2, 3, 4]`
- `plusTwo([4, 4, 7], [9, 9, 9, 2, 2]) → [4, 4, 2, 2]`
- `plusTwo([9, 2, 1, 5, 10], [3, 4]) → [9, 2, 3, 4]`

8 THE PLUS TWO PROBLEM

Given 2 int arrays, each length 2 or greater, return a new array length 4 containing the last two elements from a followed by the first two elements from b.

- `plusTwo([1, 2, 3, 4], [5, 3, 4]) → [3, 4, 5, 3]`
- `plusTwo([4, 4, 7], [9, 9, 9, 2, 2]) → [4, 7, 9, 9]`
- `plusTwo([9, 2, 1, 5, 10], [3, 4]) → [5, 10, 3, 4]`

9 THE MAKE MIDDLE PROBLEM

Given an array of ints of even length, return a new array length 2 containing the middle two elements from the original array. The original array will be length 2 or more.

- `makeMiddle([1, 2, 3, 4]) → [2, 3]`
- `makeMiddle([7, 1, 2, 3, 4, 9]) → [2, 3]`
- `makeMiddle([1, 2]) → [1, 2]`

10THE BIGGER TWO PROBLEM

Start with 2 int arrays, a and b, each length 2. Consider the sum of the values in each array. Return the array which has the largest sum. In event of a tie, return a.

- `biggerTwo([1, 2], [3, 4]) → [3, 4]`
- `biggerTwo([3, 4], [1, 2]) → [3, 4]`
- `biggerTwo([1, 1], [1, 2]) → [1, 2]`

11 THE FIX 23 PROBLEM

Given an int array length 3, if there is a 2 in the array immediately followed by a 3, set the 3 element to 0. Return the changed array.

- `fix23([1, 2, 3]) → [1, 2, 0]`
- `fix23([2, 3, 5]) → [2, 0, 5]`
- `fix23([1, 2, 1]) → [1, 2, 1]`

12THE MAKE LAST PROBLEM

Given an int array, return a new array with double the length where its last element is the same as the original array, and all the other elements are 0. The original array will be length 1 or more. Note: by default, a new int array contains all 0's.

- `makeLast([4, 5, 6]) → [0, 0, 0, 0, 0, 6]`
- `makeLast([1, 2]) → [0, 0, 0, 2]`
- `makeLast([3]) → [0, 3]`

13 THE MAKE ENDS PROBLEM

Given an array of ints, return a new array length 2 containing the first and last elements from the original array. The original array will be length 1 or more.

- `makeEnds([1, 2, 3]) → [1, 3]`
- `makeEnds([1, 2, 3, 4]) → [1, 4]`
- `makeEnds([7, 4, 6, 2]) → [7, 2]`

14THE MIDDLE WAY PROBLEM

Given 2 int arrays, a and b, each length 3, return a new array length 2 containing their middle elements.

- `middleWay([1, 2, 3], [4, 5, 6]) → [2, 5]`
- `middleWay([7, 7, 7], [3, 8, 0]) → [7, 8]`
- `middleWay([5, 2, 9], [1, 4, 5]) → [2, 4]`

15THE SUM TWO PROBLEM

Given an array of ints, return the sum of the first 2 elements in the array. If the array length is less than 2, just sum up the elements that exist, returning 0 if the array is length 0.

- `sum2([1, 2, 3]) → 3`
- `sum2([1, 1]) → 2`
- `sum2([1, 1, 1, 1]) → 2`

16THE MAKE END THREE PROBLEM

Given an array of ints, return the sum of the first 2 elements in the array. If the array length is less than 2, just sum up the elements that exist, returning 0 if the array is length 0.

- `sum2([1, 2, 3]) → 3`
- `sum2([1, 1]) → 2`
- `sum2([1, 1, 1, 1]) → 2`

17 THE REVERSE THREE PROBLEM

Given an array of ints length 3, return a new array with the elements in reverse order, so {1, 2, 3} becomes {3, 2, 1}.

- `reverse3([1, 2, 3]) → [3, 2, 1]`
- `reverse3([5, 11, 9]) → [9, 11, 5]`
- `reverse3([7, 0, 0]) → [0, 0, 7]`

18 THE ROTATE LEFT THREE PROBLEM

Given an array of ints length 3, return an array with the elements "rotated left" so {1, 2, 3} yields {2, 3, 1}.

- `rotateLeft3([1, 2, 3]) → [2, 3, 1]`
- `rotateLeft3([5, 11, 9]) → [11, 9, 5]`
- `rotateLeft3([7, 0, 0]) → [0, 0, 7]`